

Rust Custom Iterators and Pipelines

Focused revision sheet for the RAG item: custom iterators and more involved iterator pipelines over HashMap and Vec.

Description: What You Need To Know

1. Iterators are lazy

Iterator adapters such as `map` and `filter` describe work, but they do not run until a consuming operation asks for values.

```
let values = vec![1, 2, 3, 4];

let doubled = values.iter().map(|n| *n * 2);
let total: i32 = doubled.sum();
```

`map` creates a new iterator. `sum` consumes it and actually performs the work.

2. `iter`, `iter_mut`, and `into_iter`

- `iter()` yields shared references such as `&T`.
- `iter_mut()` yields mutable references such as `&mut T`.
- `into_iter()` consumes the collection and yields owned values when possible.

```
let mut values = vec![1, 2, 3];

for n in values.iter() {
    println!("{}", n);
}

for n in values.iter_mut() {
    *n += 1;
}

for n in values.into_iter() {
    println!("{}", n);
}
```

After `into_iter()` consumes a vector of owned values, the original vector cannot be used again.

3. Common adapters

Adapters transform an iterator into another iterator. They are usually chained before a final consuming operation.

```
let values = vec![1, 2, 3, 4, 5, 6];

let result: Vec<i32> = values
    .iter()
    .filter(|n| **n % 2 == 0)
    .map(|n| *n * *n)
    .collect();

// result is [4, 16, 36]
```

With `iter()`, the closure often receives references. That is why the filter example uses `**n`: the closure argument is a reference to an iterator item that is itself a reference.

4. Useful consumers

- `collect()`: builds a collection from iterator items.
- `sum()`: adds numeric items.
- `count()`: counts items.
- `any()`: checks whether at least one item matches.
- `all()`: checks whether every item matches.
- `fold()`: accumulates a result using a closure.

```
let values = vec![1, 2, 3, 4];

let total: i32 = values.iter().sum();
let big = values.iter().any(|n| *n > 3);
let product = values.iter().fold(1, |acc, n| acc * *n);
```

5. Pipelines over strings

String and text exercises often combine splitting, normalising, filtering, and collecting.

```
let text = "Rust, rust! C?";

let words: Vec<String> = text
    .split_whitespace()
    .map(|word| {
        word.trim_matches(|ch: char| !ch.is_alphanumeric())
            .to_lowercase()
    })
    .filter(|word| !word.is_empty())
    .collect();

// ["rust", "rust", "c"]
```

Each stage does one job: split into rough words, trim punctuation, lower-case, remove empty strings, then collect.

6. Pipelines over HashMap

A HashMap iterator yields key-value pairs. With `iter()`, those pairs are references.

```
use std::collections::HashMap;

let mut scores = HashMap::new();
scores.insert("red", 12);
scores.insert("blue", 7);
scores.insert("green", 15);

let mut high: Vec<(&str, i32)> = scores
    .iter()
    .filter(|(_, score)| **score >= 10)
    .map(|(team, score)| (*team, *score))
    .collect();

high.sort_by_key(|(_, score)| *score);
```

The filter closure gets references from the map iterator. The map step copies out the borrowed key and integer score so the resulting vector has simpler values.

7. Custom iterator structure

A custom iterator is a type that stores iteration state and implements the `Iterator` trait. The important method is `next`.

```
struct Counter {
    current: u32,
    end: u32,
}

impl Counter {
    fn new(end: u32) -> Counter {
        Counter { current: 0, end }
    }
}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        if self.current >= self.end {
            None
        } else {
            let value = self.current;
            self.current += 1;
            Some(value)
        }
    }
}
```

`next` returns `Some(item)` while there is another value and `None` when iteration is finished.

```
let values: Vec<u32> = Counter::new(4).collect();
// [0, 1, 2, 3]
```

8. Custom iterators still use adapters

Once a type implements `Iterator`, it can use the standard iterator adapters and consumers.

```
let total: u32 = Counter::new(10)
    .filter(|n| *n % 2 == 0)
    .map(|n| n * n)
    .sum();
```

This pipeline keeps even numbers from 0 to 9, squares them, then sums the squares.

Practice Questions

- 1 Explain why `values.iter().map(|n| *n * 2)` alone does not produce a vector.
- 2 Choose `iter`, `iter_mut`, or `into_iter` for reading, modifying in place, and consuming a vector.
- 3 Write a pipeline that keeps even numbers from a vector and collects their squares into a new vector.
- 4 Use `fold` to multiply all numbers in a vector.
- 5 Given a sentence, split it into words, lowercase them, and collect into `Vec<String>`.
- 6 Given a `HashMap<String, i32>`, collect entries with score at least 10 into a vector.
- 7 Write the struct fields needed for a simple counter iterator from 0 up to but not including an end value.
- 8 Implement the `next` method for that counter iterator.
- 9 Explain why `next` returns `Option`.
- 10 Use your custom counter with `filter`, `map`, and `sum`.

Mark Scheme

Use this after attempting the questions. Strong answers should distinguish adapters from consumers and ownership from borrowing.

- 1 `map` is lazy and returns another iterator. Use a consumer such as `collect`, `sum`, or a `for` loop to run it.
- 2 Use `iter` for shared reads, `iter_mut` for in-place mutation, and `into_iter` when consuming owned values.
- 3 Expected pattern: `values.iter().filter(|n| **n % 2 == 0).map(|n| *n * *n).collect::<Vec<_>>()`.
- 4 Expected pattern: `values.iter().fold(1, |acc, n| acc * *n)`.
- 5 Expected stages: `split_whitespace`, map to lower-case or clean, then `collect`.
- 6 Expected: iterate with `map.iter()`, filter by score, and collect copied or cloned key-value pairs into a vector.
- 7 Expected fields include current position and end limit, such as `current: u32` and `end: u32`.
- 8 Expected: return `None` when current reaches end; otherwise save current, increment it, and return `Some(value)`.
- 9 `Option` represents either a next item with `Some` or finished iteration with `None`.
- 10 Expected pattern: `Counter::new(10).filter(...).map(...).sum()`.

Common Mistakes

- Forgetting that adapters are lazy until a consumer runs.
- Using `into_iter` and then trying to use the consumed vector again.
- Getting confused by references in `filter` closures.
- Returning a plain value from `next` instead of `Option<Item>`.
- Forgetting to update iterator state before returning `Some`.